# ZetaERP — Manual Bug-Testing SOP

## Part 8 — Subscriptions (tenant → customer recurring billing) & My Subscription (tenant → ZetaERP platform)

| | |
|---|---|
| **Document** | QA-SOP-08 — Subscriptions & My Subscription |
| **Product** | ZetaERP / Zenith (multi-tenant POS / ERP) |
| **Environment under test** | Production — https://erp.zetaclantech.com |
| **Scope of this part** | Two related recurring-billing surfaces: **(A) the Subscriptions module** ( `/apps/zenith/subscriptions/…` ) — a *tenant billing its own customers* on a recurring GST subscription (plans, discounts, lifecycle, generated GST invoices); and **(B) "My Subscription"** ( `/apps/zenith/my-subscription` ) — the *tenant's own subscription to the ZetaERP SaaS platform* (plan, usage vs limits, module access, trial/past-due, Razorpay payments, upgrade/downgrade). Cross-references the super-admin **SaaS Admin** ( `/apps/zenith/saas-admin` ) where relevant. |
| **Audience** | Manual QA tester (no code knowledge assumed; basic GST literacy helps for Part A) |
| **Document owner** | Platform / Solo-dev |
| **Status** | Active |
| **Last updated** | 2026-07-01 |

**CONTENTS**


1. Purpose
2. Critical rules for the tester (read before starting)
3. Test environment & preconditions
4. Test data
5. Conventions & severity
6. Flow map
7. Plans & Discounts
8. Create Subscription (A→Z) + lifecycle
9. Generate Invoice & GST correctness (the money + tax) — do the math
10. Part A multi-tenant isolation (test explicitly — known gaps)
11. My Subscription dashboard
12. Usage vs plan limits
13. Upgrade / Downgrade / Cancel
14. Module access gating (plan → modules)
15. Payments (Razorpay) & lifecycle automation
16. Super-admin SaaS Admin (isolation boundary — brief)
17. Cross-flow / regression & defect template
18. Areas to probe hardest (attention map)
19. Teardown (mandatory)


## 1. Purpose

This SOP tells a tester **exactly how to manually test both recurring-billing surfaces A→Z**, including edge and UI cases:

- **Part A — Subscriptions:** create plans/discounts, create a customer subscription, run it through its lifecycle (trial → active → pause/resume → renew → change plan → cancel/expire), **generate the recurring GST invoice**, and verify the **CGST/SGST/IGST** is correct on the *actual invoice*, discounts apply, and everything is **isolated per tenant**.
- **Part B — My Subscription:** the tenant's view of what they pay ZetaERP — plan details, **usage vs plan limits**, **included modules** (and that out-of-plan modules are blocked), **pending platform invoices + Razorpay payment**, payment history, **upgrade/downgrade**, cancel, and the **trial-expiry / past-due / suspension** automation.

> **Both surfaces produce money records** — GST tax invoices to customers (Part A) and platform charges to tenants (Part B). Do the GST math by hand, and never test against a live tenant's real subscriptions, invoices, or payments.

---

## 2. Critical rules for the tester (read before starting)

> ⚠️ **This is a LIVE commercial billing system.**

1. **NEVER create, edit, bill, renew, cancel, or pay** a real subscription/invoice/payment on the live flagship tenant **MARUTHI (org `100000`)**, any other live tenant, or the platform billing org `ZETAERP`. Use a **throwaway tenant** you control.
2. **Never make a real payment.** Razorpay is a **real gateway**. Only use Razorpay **test keys / test cards** if the environment is configured for test mode — confirm with the dev first. Never pay a real invoice with a real card.
3. **Multi-tenant isolation is a top-priority surface here** (§10, §16). Subscriptions, plans, discounts, and invoices must be scoped to the active org. **Part A has known cross-tenant gaps (§16) — test them explicitly.**
4. **GST invoices are legal records** — a wrong CGST/SGST/IGST split, a missing head-split, or a shared invoice-number series is a compliance issue (§9). Verify by hand.
5. **Clean up every test artefact** (plan, discount, subscription, invoice, payment, throwaway tenant) — or hand the dev the list. Do not leave test subscriptions that a billing job could later invoice.
6. **Record inputs + by-hand expected split vs actual** for every GST case; a bare "looks right" is not acceptable for a billing case.

---

## 3. Test environment & preconditions

**Part A — Subscriptions** — `/apps/zenith/subscriptions/…` (lowercase, flat — screens are directly under `subscriptions/`, **not** under a nested `Subscriptions/`). `/apps` Bootstrap. **Note: `subscriptions` is a plan-gated module** — the test tenant's SaaS plan must include it, or the page redirects to the Dashboard (see Part B §14).

| Screen (URL) | Purpose |
|---|---|
| `/apps/zenith/subscriptions` (→ `AllSubscriptions`) | List + "New Subscription" dialog |
| `/apps/zenith/subscriptions/EditSubscription?subscriptionId=` | Detail + lifecycle actions + invoices + activity log (hidden from menu) |
| `/apps/zenith/subscriptions/Plans` | Create/edit subscription plans |
| `/apps/zenith/subscriptions/Discounts` | Coupon codes |
| `/apps/zenith/subscriptions/Invoices` | Generated subscription invoices |

**Part B — My Subscription** — `/apps/zenith/my-subscription` (tenant view, `/apps` Bootstrap). Super-admin: `/apps/zenith/saas-admin` (**ADMIN-only** — 403/blocked otherwise). Public: `/signup`, `/get-started`, `/razorpay-webhook`.

**Set up on a throwaway tenant:** a customer with a **GSTIN in the same state** (→ intra) and one in a **different state** (→ inter) for Part A GST tests; the tenant's SaaS plan should include the **subscriptions** module. For Part B, know the tenant's current SaaS plan + limits.

**Browsers:** full pass on Desktop Chrome; spot-check Edge/Safari/mobile. **Tools:** a **calculator** (GST by hand), DevTools (Console/Network), the state-code table (01–38), and Razorpay **test** cards only.

---

## 4. Test data

| Label | Use |
|---|---|
| Throwaway tenant | Non-MARUTHI, non-ZETAERP org for all testing |
| `PLAN_M` | A monthly plan (e.g. ₹1000/mo, 18% GST, HSN 998314, trial 0) |
| `PLAN_T` | A plan with a **trial** (e.g. trialDays 14) + a **setup fee** |
| `CUST_SAME` / `CUST_OTHER` | Customers with GSTIN in the tenant's state / another state |
| `CUST_B2C` | Customer with no GSTIN |
| `DISC_PCT` / `DISC_FLAT` | A % discount and a flat discount coupon |
| Amounts | ₹1000/mo @18% → taxable 1000; intra CGST 90 + SGST 90; inter IGST 180 |

## 5. Conventions & severity

**Case Type:** `FUNC` · `GST` (tax correctness) · `LIFE` (lifecycle) · `EDGE` · `UI` · `A11Y` · `SEC` · `ISO` · `PAY` (payment) · `PERF` .

| Sev | Meaning | Example |
|---|---|---|
| **S1 — Critical** | Cross-tenant read/write of subscriptions/invoices; wrong GST on an issued invoice; a real (non-test) payment charged; billing another tenant's subscription | tenant A opens/renews tenant B's subscription by id; IGST charged on an intra-state renewal |
| **S2 — High** | A billing flow broken or clearly wrong | Generate Invoice leaves CGST/SGST/IGST null; discount not reflected on the invoice; downgrade below current usage allowed silently |
| **S3 — Medium** | Wrong/loose behaviour, limited impact | one-time plan doesn't advance dates; upgrade has no proration; trial date off by a day |
| **S4 — Low** | Cosmetic / minor UX | usage bar mis-coloured; plan card spacing; label typo |

**Logging a defect** (template §15): Case ID, steps, **inputs + by-hand expected vs actual**, tenant(s), seller/buyer state codes (Part A), browser, screenshot, Console/Network. Evidence **mandatory** for GST, ISO, SEC, PAY.

## 6. Flow map

```
PART A (tenant → customer)                    PART B (tenant → ZetaERP)
Plans/Discounts → Create Subscription         Signup/Provision → SaasSubscription (trial/active)
   (trial? setupFee? discount?)                  |
      | status SubTrial/SubActive                ├─ My Subscription: plan, usage vs limits, modules
      ▼                                          ├─ Upgrade/Downgrade (immediate swap, no proration)
Lifecycle: Activate/Pause/Resume/Cancel          ├─ Pending invoice → Razorpay Pay Now → SiPaid
Renew (advance dates)  ⊥  Generate Invoice (bill)└─ Crons: trial-expiry, renewal-invoice, overdue-suspend
      |  ⚠ these two are DECOUPLED and MANUAL
      ▼
SubscriptionInvoice (GST)   ⚠ generate#Invoice uses FLAT tax (no CGST/SGST/IGST split)
```

# PART A — SUBSCRIPTIONS (tenant bills its customers)

## 7. Plans & Discounts

**Plans** — `/apps/zenith/subscriptions/Plans` (entity `SubscriptionPlan` : planName, planCode, billingFrequency Monthly/Quarterly/HalfYearly/Annually/OneTime, billingAmount, setupFee, trialDays, taxRate, productTaxCode/HSN, maxUsers, autoRenew).

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| SB-001 | FUNC | Create a plan | New plan (₹1000/mo, 18%, HSN 998314, trial 0) → Save | Persists; org-scoped (has your organizationPartyId) | S2 |
| SB-002 | FUNC | Plan with trial + setup fee | Create `PLAN_T` (trialDays 14, setupFee ₹500) | Persists | S3 |
| SB-003 | EDGE | Free / zero plan | billingAmount 0 | Allowed; totals compute to 0 downstream | S3 |
| SB-004 | ISO | Plans scoped | List Plans as tenant A | Only A's plans **plus the global seed plans** (seed rows have no org → appear for everyone — record/flag) | S2 |

**Discounts** — `/apps/zenith/subscriptions/Discounts` ( `SubscriptionDiscount` : discountCode, type Percentage/Flat/TrialExtend, discountValue, maxUses, validFrom/To, applicablePlanId).

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| SB-005 | FUNC | Create % + flat coupons | `DISC_PCT` (10%), `DISC_FLAT` (₹100) | Persist, org-scoped | S3 |
| SB-006 | EDGE | Validity window / max uses | Coupon outside its date window, or past `maxUses` | Not applied at create time | S3 |
| SB-007 | EDGE | **Discount > price** | Flat ₹2000 on a ₹1000 plan (or % > 100) | ⚠ No floor in code → **negative** billingAmount/GST/total possible. Record actual; a negative invoice is a bug | S2 |

## 8. Create Subscription (A→Z) + lifecycle

**Screen:** `/apps/zenith/subscriptions` → "New Subscription" (fields: `customerPartyId` — **free-text party id, no picker**, `planId` , `paymentMethodId` , `discountCode` , `notes` ). Service `create#Subscription` (requires an active org; generates org-scoped `SUB/FY/NNNNN` ; computes trial/end/next-bill dates; calls `calculate#SubscriptionTax` ; applies discount).

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| SB-010 | FUNC | Create (no trial) | New subscription, `PLAN_M` , `CUST_SAME` | Created, status **SubActive**, `SUB/FY/NNNNN` number, next-bill date = start + 1 month; GST computed | S2 |
| SB-011 | FUNC | Create with trial | Use `PLAN_T` (trial 14) | Status **SubTrial**, trialEndDate = start + 14d | S3 |
| SB-012 | FUNC | Create with discount | Apply `DISC_PCT` | billingAmount reduced; **GST recomputed on the discounted base** | S2 |
| SB-013 | EDGE | Bad customer id | Enter a non-existent / another tenant's party id in `customerPartyId` | Record actual — is the id validated / org-checked? (free-text, no picker) | S2 |
| SB-014 | EDGE | Requires active org | (Admin with no active org) create | Errors clearly ("no active organization") | S3 |
| SB-015 | LIFE | Activate | On a SubTrial subscription → **Activate** | → SubActive; activity log `SaCreated` /activate entry | S2 |
| SB-016 | LIFE | Pause / Resume | SubActive → **Pause** → SubPaused → **Resume** → SubActive | State transitions correct; logged | S2 |
| SB-017 | LIFE | Cancel | → **Cancel** (with reason) | → SubCancelled, cancelReason + cancelDate stored; cannot re-cancel | S2 |
| SB-018 | LIFE | Renew | SubActive/PastDue → **Renew** | endDate + nextBillingDate advanced by frequency; lastBillingDate=now; ⚠ **no invoice created** (renew ≠ bill) | S2 |
| SB-019 | LIFE | Change plan | SubActive/Trial → **Change Plan** | planId + billingAmount swapped; ⚠ GST recomputed **flat (no head split), no proration**; logged Upgraded/Downgraded by price | S2 |
| SB-020 | EDGE | One-time plan | Plan with `OneTime` frequency | ⚠ end-date logic has **no OneTime branch** → endDate == start (no advance). Record actual | S3 |

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| SB-021 | LIFE | Action buttons by status | Inspect the detail page at each status | Buttons appear only when valid (Generate Invoice = **SubActive only**, etc.) | S3 |

## 9. Generate Invoice & GST correctness (the money + tax) — do the math

**Service** `generate#Invoice` creates a `SINV/FY/NNNNN` invoice (setupFee charged only on the first). ⚠ **Known gaps to verify (they are the point of this section):**

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| SB-030 | FUNC | Generate first invoice | SubActive subscription → **Generate Invoice** | `SINV/FY/NNNNN` created; **setup fee included** on the first only | S2 |
| SB-031 | GST | **Intra-state split on the invoice** | Customer in the tenant's state, ₹1000 @18% | Expect CGST 90 + SGST 90 on the **invoice**. ⚠ `generate#Invoice` uses **FLAT tax** and leaves `cgstAmount/sgstAmount/igstAmount/placeOfSupply` **NULL** — the split computed on the Subscription at create-time is **not carried to the invoice**. Record actual — a GST invoice with no head split is non-compliant | **S1** |
| SB-032 | GST | **Inter-state split on the invoice** | Customer in another state | Expect IGST 180 on the invoice; same null-split gap likely | **S1** |
| SB-033 | GST | Discount on the invoice | Subscription created with a discount | ⚠ `generate#Invoice` hardcodes `discountAmount = 0` → the invoice may **not reflect the discount**. Record actual | S2 |
| SB-034 | GST | **Invoice number is org-scoped?** | Generate invoices on **two** tenants; compare `SINV` numbers | ⚠ The SINV series is **NOT org-scoped** (shared across tenants) → gaps/overlaps in each tenant's sequence — a GST consecutiveness problem. (Contrast: SUB numbers ARE org-scoped.) | S2 |
| SB-035 | GST | Tax-exclusive base | Verify the base | Prices are tax-**exclusive**; GST added on top; total = base + tax | S2 |
| SB-036 | FUNC | Invoices screen | `/apps/zenith/subscriptions/Invoices` → open a generated invoice | Shows period, amounts, status (Draft/Sent/Paid/Overdue…); status editable | S3 |
| SB-037 | EDGE | **No recurring billing job** | Wait for a due subscription to auto-bill | ⚠ There is **no scheduled/cron job** — billing is **entirely manual** (open each sub, click Generate + Renew). Confirm nothing auto-generates; this is a functional gap to raise | S2 |

## 10. Part A multi-tenant isolation (test explicitly — known gaps)

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| ISO-001 | ISO | List scoped | Tenant A lists subscriptions/invoices | Only A's rows | S1 |
| ISO-002 | SEC | **View another tenant's subscription** | As tenant A, open `EditSubscription?subscriptionId=<B's id>` | ⚠ Should be blocked. Detail load has **no org check** → A may **see B's subscription, invoices, activity**. Confirm + flag (Critical IDOR) | **S1** |
| ISO-003 | SEC | **Mutate/bill another tenant's subscription** | As A, call **Renew / Change Plan / Generate Invoice** on B's subscription | ⚠ These 3 have **no ownership guard** → A could renew/re-plan/bill B's subscription. (Activate/Pause/Resume/Cancel **are** guarded.) Confirm + flag | **S1** |
| ISO-004 | ISO | Plans/discounts scoped | A lists Plans/Discounts | Only A's + the global seed rows (flag the seeds) | S2 |

# PART B — MY SUBSCRIPTION (tenant → ZetaERP platform)

## 11. My Subscription dashboard

**Screen:** `/apps/zenith/my-subscription` ( `TenantDashboard` , resolves the tenant from **ACTIVE_ORGANIZATION** — never a client-supplied id). Entities `SaasTenant` , `SaasSubscription` , `SaasPlan` , `SaasInvoice` , `SaasPayment` .

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| MS-001 | FUNC | Dashboard renders | Open My Subscription | Subscription Details (plan, status, next billing, trial end), Usage, Modules, Pending Payments, History, Available Plans — all for **your** tenant | S2 |
| MS-002 | FUNC | Status shows correctly | Trial vs Active vs Past-due vs Cancelled | Correct status + dates (trialEndDate, renewalDate) | S2 |
| MS-003 | UI | Styling on /apps | Inspect the Quasar-styled cards | The screen injects its own `<style>` (Quasar classes are no-ops on /apps otherwise) — cards render, not raw | S3 |

## 12. Usage vs plan limits

Usage card shows users / POS terminals / products / monthly txns / stores / loyalty / promotions vs the plan's limits (**0 = unlimited**).

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| MS-010 | FUNC | Usage counts correct | Compare each metric to reality | Counts match; unlimited (0) shows as ∞/no cap | S2 |
| MS-011 | UI | Over-limit warning | A metric above the plan limit | Clear over-limit indicator (colour/warning) | S3 |
| MS-012 | EDGE | Free (STARTER) plan | Tenant on STARTER (₹0) | Limits enforced/shown per STARTER; renewal invoices are ₹0 | S3 |

## 13. Upgrade / Downgrade / Cancel

**Transitions:** `changePlan` → `SaasServices.change#Plan` (immediate swap); `cancelSubscription` → `cancel#Subscription` .

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| MS-020 | FUNC | Upgrade plan | Pick a higher plan → Upgrade | tenant.planId swapped, new SubActive subscription; limits/modules update | S2 |
| MS-021 | EDGE | **No proration / no invoice** | Upgrade mid-cycle | ⚠ Immediate swap, **no proration and no invoice generated** — record actual (revenue/UX consideration) | S3 |
| MS-022 | EDGE | **Downgrade below current usage** | On a tenant over the target plan's limits, downgrade | ⚠ `change#Plan` **does not call** `check#UsageLimits` → the downgrade **succeeds despite being over limit** (limits are display-only). Confirm + flag | S2 |
| MS-023 | LIFE | Cancel subscription | Cancel | autoRenew off; tenant stays active until period end (unless immediate); status reflects cancellation | S2 |

## 14. Module access gating (plan → modules)

The central gate (server-side in `ZenithApp` ) redirects a tenant away from a module **not** in its plan (fail-open — only blocks positively-excluded modules; ADMIN exempt). `subscriptions` (Part A!) is one of the gateable modules.

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| MS-030 | FUNC | Included modules shown | My Subscription → Modules | Chips match the plan's allowed modules | S2 |
| MS-031 | SEC | **Out-of-plan module blocked (server-side)** | Direct-URL to a module **not** in your plan (e.g. `/apps/zenith/subscriptions` when subscriptions isn't included) | Redirected to the Dashboard with a message — **not** just hidden from the menu | S1 |
| MS-032 | SEC | Enterprise "ALL" access | Tenant on ENTERPRISE (module `ALL` ) | Every module reachable | S3 |

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| MS-033 | SEC | Suspended tenant blocked | A suspended tenant opens any screen | "Account Suspended" block screen | S1 |

## 15. Payments (Razorpay) & lifecycle automation

**Real Razorpay** ( `create#RazorpayOrder` / `verify#RazorpayPayment` — HMAC verify + replay/idempotency guards). **Only test with Razorpay test keys/cards; never a real payment.**

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| MS-040 | PAY | Pending invoice + Pay Now | A pending SaaS invoice → **Pay Now** | Razorpay checkout opens (test mode); on success → invoice **SiPaid**, payment recorded, tenant reactivated if suspended | S1 |
| MS-041 | SEC | **Pay only your own invoice** | Try to pay an invoice belonging to another tenant (tamper the id) | Rejected — the transition re-checks `invoice.tenantId == my tenant` | S1 |
| MS-042 | PAY | Razorpay not configured | If `RAZORPAY_KEY_ID/SECRET` absent | Order creation **errors clearly** (feature unavailable) — no silent fake success | S2 |
| MS-043 | PAY | Idempotent / replay-safe | Re-post the same payment / order | No double-record; signature/replay guard holds | S2 |
| MS-044 | FUNC | Payment history | View history | Last payments listed for **your** tenant only | S3 |
| MS-045 | LIFE | Trial expiry (cron) | (Dev-assisted) trial past its end | Daily job: SubTrial → SubExpired, tenant Suspended | S2 |
| MS-046 | LIFE | Renewal invoice (cron) | (Dev-assisted) renewal due + autoRenew | Daily job creates a `SAAS-yyyyMM-NNNN` invoice; **GST correct** (routes through the India engine, ZETAERP as supplier → proper CGST/SGST/IGST); idempotent (no duplicate for the same period) | S1 |
| MS-047 | LIFE | Overdue → suspend (cron) | (Dev-assisted) invoice past due + grace | SiSent→SiOverdue, then tenant Suspended + subscription SubPastDue after the grace days | S2 |
| MS-048 | GST | Super-admin manual invoice | (ADMIN) create a SaaS invoice via SaaS Admin → Invoices | ⚠ Only the **cron** path sets the CGST/SGST/IGST head split; a manual/admin-created invoice may not. Verify | S2 |

## 16. Super-admin SaaS Admin (isolation boundary — brief)

**Screen:** `/apps/zenith/saas-admin` (**ADMIN only**).

| ID | Type | Title | Steps | Expected result | Sev |
|---|---|---|---|---|---|
| ISO-010 | SEC | Admin gate | Open `/apps/zenith/saas-admin` as a **non-admin tenant** | Blocked / 403 | S1 |
| ISO-011 | ISO | Tenant view scoped | As a tenant, confirm My Subscription only ever shows your own tenant | Resolved from ACTIVE_ORGANIZATION; no other tenant's data reachable | S1 |
| ISO-012 | SEC | Underlying services take tenantId | (Dev-assisted) note `change#Plan` / `cancel#Subscription` /payment services accept `tenantId` with no internal owner check | Safe **only** because the tenant transitions resolve tenantId server-side — verify no path exposes a client-supplied tenantId | S2 |

## 17. Cross-flow / regression & defect template

| ID | Type | Title | Expected result | Sev |
|---|---|---|---|---|
| XF-001 | GST | Subscription invoice vs tax engine | A subscription invoice's CGST/SGST/IGST matches what the tax engine (Part 6) would compute for the same customer/amount — including the head split | S1 |
| XF-002 | ISO | Two-tenant isolation, both parts | Nothing tenant A configures/subscribes/pays is visible or mutable by tenant B (Part A **and** Part B) | S1 |

| ID | Type | Title | Expected result | Sev |
|---|---|---|---|---|
| XF-003 | LIFE | Renew ⊥ Generate coupling | Renewing advances dates but bills nothing; generating bills but doesn't advance — confirm the operator must do both, and flag the decoupling | S2 |
| XF-004 | PAY | Pay → reactivate | Paying an overdue platform invoice reactivates the suspended tenant and settles **only that** invoice (not all) | S1 |

```
Defect ID:        ZE-SUB-____
Linked Case ID:   SB-0__ / MS-0__ / ISO-0__ / XF-0__
Title / Severity / Type:
Environment:      Prod | <browser> | tenant: <org> (NEVER MARUTHI/ZETAERP)
Inputs:           plan, amount, customer state, discount, billing frequency, trial
Expected (by hand for GST): taxable, CGST, SGST, IGST, discount, total, invoice no.
Actual:           (same) — attach screenshot + Console/Network
Records touched:  subscription / invoice / payment ids
```

## 18. Areas to probe hardest (attention map)

1. **Part A cross-tenant access (ISO-002/003)** — `EditSubscription` loads by id with **no org check**, and **Renew / Change Plan / Generate Invoice** have **no ownership guard**. A tenant may view — and *bill* — another tenant's subscription. This is the top risk; confirm and flag as Critical.
2. **GST head-split missing on the actual invoice (SB-031/032/033)** — `generate#Invoice` uses **flat tax**, leaves CGST/SGST/IGST/placeOfSupply **NULL**, and ignores the discount (hardcoded 0). The split computed on the Subscription at create-time is **not** carried to the `SubscriptionInvoice` — a non-compliant GST invoice.
3. **Subscription invoice numbering not org-scoped (SB-034)** — the `SINV` series is shared across tenants, breaking per-tenant GST consecutiveness.
4. **No recurring-billing automation for Part A (SB-037, XF-003)** — billing is entirely manual, and Renew vs Generate are decoupled (dates advance without a bill, or a bill without advancing dates).
5. **Downgrade below current usage not blocked (MS-022)** — `change#Plan` never calls `check#UsageLimits`.
6. **Module gating is server-side and fail-open (MS-031/033)** — verify an out-of-plan module (including `subscriptions`) is actually **redirected**, not merely hidden; suspended tenants are blocked.
7. **Razorpay is real (MS-040/041/042)** — never make a real payment; verify the own-invoice check, the not-configured error, and replay/idempotency.
8. **Discount > price → negative (SB-007)** and **one-time plan no date advance (SB-020)** — edge maths.
9. **SaaS renewal GST is correct via the cron path (MS-046)** but a manual/admin invoice may miss the split (MS-048) — verify both.

## 19. Teardown (mandatory)

1. **List every test artefact** — plans, discounts, subscriptions, subscription invoices, SaaS invoices, payments, and any throwaway tenant — with ids.
2. **Do not leave active test subscriptions** that a (future) billing job could invoice; cancel/delete them.
3. **Refund/void any test payment** made in Razorpay test mode; never leave a real charge.
4. **Delete** test plans/discounts/subscriptions/invoices (or hand the dev the list). Tear down any throwaway tenant (see the Auth/Signup teardown recipe).
5. Confirm you **never touched MARUTHI** (org `100000`), **ZETAERP**, or any live tenant's subscriptions/invoices/payments.
6. Attach the run sheet(s) + defect ids to this part's report.

*End of Part 8 — Subscriptions & My Subscription. Related: Part 6 — Tax (the GST engine subscription invoices should use); Part 2 — Contacts (the customer parties billed); Auth/Signup (tenant provisioning for the SaaS side). Before any live subscription billing, triage with the dev: the Part-A cross-tenant IDOR + missing ownership guards, the flat-tax/no-head-split on generate#Invoice, the shared SINV numbering, and the absence of a recurring-billing job.*